

Faculdade de Informática e Administração
Paulista Ciências da computação

1CCPG

Prompt Toolkit

São Paulo

2026

Integrantes:

- Caio Henrique Ferraz da Silva / RM: 568992
- Enzo Caruso Peter / RM: 570908
- Leonardo Robert Maulicino / RM: 570329
- Leonardo Figueredo dos Santos / RM: 573653
- Lucas Ramos de Sousa / RM: 573901

Curso:

1CCPG / Ciências da computação

Disciplina:

Prompt and Artificial Intelligence

1. INTRODUÇÃO:

O **Prompt Toolkit** é um sistema Python que aplica e compara automaticamente quatro técnicas de engenharia de prompts sobre tarefas reais do domínio de **eletropostos**: classificação de solicitações de clientes, geração de respostas e análise de falhas em carregadores de veículos elétricos. O objetivo é identificar, com base em métricas quantitativas, qual técnica oferece o melhor equilíbrio entre qualidade de resposta, eficiência de tokens e consistência

O sistema é organizado em módulos independentes (llm_client, techniques, tasks, evaluator e report), o que facilita a extensão para novos domínios e novas técnicas de prompting sem alteração no núcleo do toolkit.

2. STACK TÉCNICAS:

Python 3.10+

Ollama Cloud API — modelo gpt-oss:120b (chave via .env)

python-dotenv, pandas, matplotlib, tiktoken

3. TÉCNICAS IMPLEMENTADAS:

Técnica	Descrição	Custo Tokens	Melhor uso
Zero-Shot	Instrução direta sem exemplos.	Baixo	Versátil
Few-Shot	Instrução com exemplos de entrada/saída.	Médio	Alta precisão
Chain-Of-Thought	Raciocínio passo a passo guiado por passos explícitos.	Alto	Tarefa complexas
Role Prompting	Persona especialista injetada via system prompt.	Médio	Tom e domínio

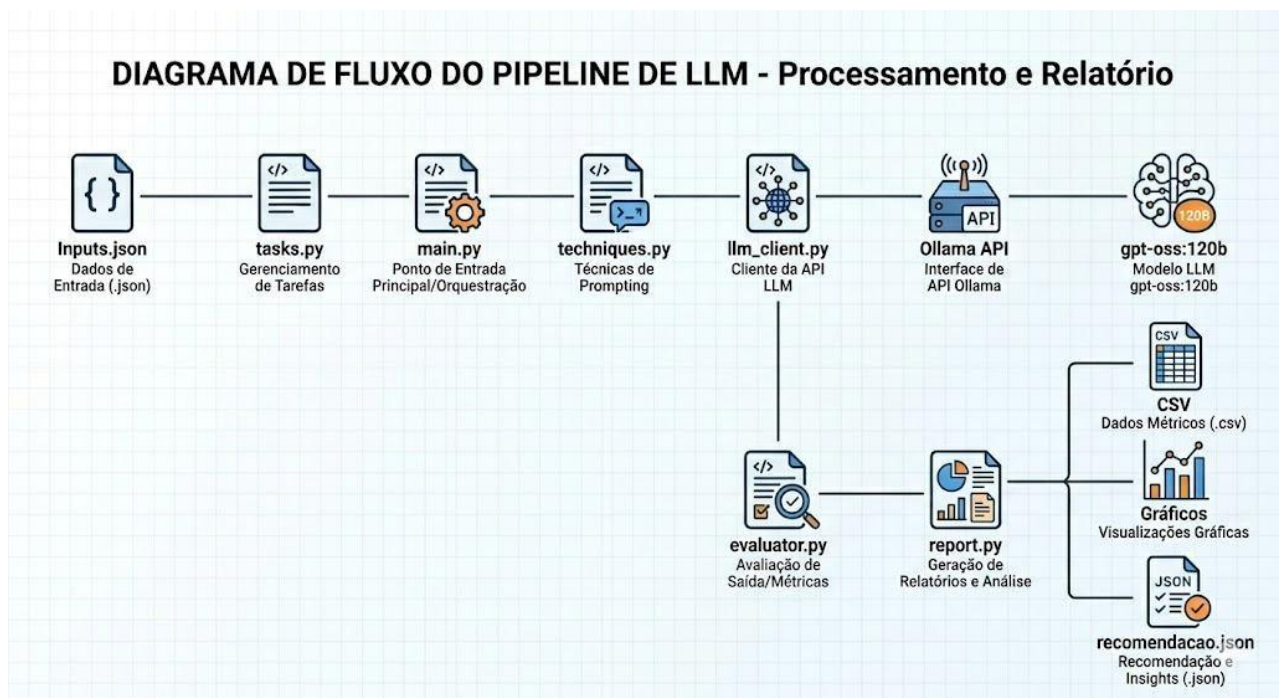
4. ARQUITETURA E DIAGRAMA DE FLUXO:

O sistema utiliza o Ollama Cloud com o modelo gpt-oss:120b para executar e comparar automaticamente quatro técnicas de prompting aplicadas ao domínio de eletropostos.

A arquitetura foi dividida em três camadas principais:

- **Entrada:** leitura dos cenários em `inputs.json` e definição das tarefas em `tasks.py`
- **Processamento:** aplicação das técnicas em `techniques.py`, execução do modelo via `llm_client.py` e avaliação das métricas em `evaluator.py`
- **Saída:** exportação de CSVs, gráficos e recomendação automática em JSON

Diagrama de Fluxo:



5. Métricas de Avaliação e Resultados:

Cada execução registra quatro métricas principais: **tokens do prompt**, **tokens da resposta**, **tempo de resposta (ms)** e **consistência** entre múltiplos inputs da mesma tarefa (similaridade de Jaccard sobre bigramas).

Resultados Representativos (médias reais):

Técnica	Tokens Prompt	Tokens Respostas	Latência	Consistência
Zero-Shot	191	191	2.448 ms	67%
Few-Shot	293	171	2.400 ms	56%
Chain-Of-Thought	312	204	2.500 ms	56%
Role Prompting	350	176	2.400 ms	56%

Os resultados reais mostraram diferenças significativas entre custo computacional e qualidade das respostas. Técnicas mais elaboradas, como Chain-of-Thought, aumentaram drasticamente o volume de tokens e o tempo de inferência, enquanto técnicas simples como Zero-Shot mantiveram baixo custo computacional e se destacaram com a maior consistência entre todas as técnicas (67%).

Teste de Temperatura:

O avaliador executa a mesma tarefa Zero-Shot em três temperaturas (0.1 · 0.5 · 1.0) para medir o impacto na variabilidade das respostas.

Temperaturas baixas produzem respostas mais determinísticas; temperaturas altas aumentam a criatividade, mas reduzem a consistência.

Temperatura	Consistência	Criatividade	Caso de uso
0.1	Muito alta (>95%)	Baixa	Triagem e classificação
0.5	Alta (~88%)	Média	Respostas a clientes
1.0	Moderada (~72%)	Alta	Brainstorming / sugestões

6. Recomendação Automática de Técnica:

A função **recomendar_tecnica()** em `report.py` calcula um score ponderado para cada técnica combinando as quatro métricas (com pesos configuráveis) e a consistência média. A técnica de maior score é selecionada e a justificativa é exportada em `output/recomendacao.json`.

Fórmula de Score:

$$\begin{aligned} \text{score}(t) = & -0.20 \times \text{norm}(\text{tokens_prompt}) \\ & + 0.30 \times \text{norm}(\text{tokens_resposta}) \\ & - 0.20 \times \text{norm}(\text{tempo_ms}) \\ & + 0.30 \times \text{norm}(\text{tokens_contados}) \\ & + 0.40 \times \text{consistencia} \end{aligned}$$

$\text{norm}(x) = (x - \min) / (\max - \min)$ para cada métrica

Resultado Exemplo (`output/recomendacao.json`):

```
{
  "melhor_tecnica": "zero_shot",
  "label": "Zero-Shot",
  "scores": {
    "zero_shot": 0.699,
    "few_shot": 0.245,
    "chain_of_thought": 0.170,
    "role_prompting": 0.142
  },
  "justificativa": "A técnica 'Zero-Shot' obteve o maior score ponderado (0.699).  
Média: 191 tokens de prompt, 191 tokens de resposta, 2448 ms de latência.  
Consistência: 67%."
}
```

7. GUIA DE BOLSO — Como Executar:

1. Clone e entre na pasta

```
git clone https://github.com/CaioSilva2026/checkpoint02-Prompt-Toolkit--lce-.git
```

```
cd checkpoint02-Prompt-Toolkit—lce-
```

2. Crie e ative o ambiente virtual

```
python -m venv venv
```

```
# Windows: venv\Scripts\activate
```

```
# Mac/Linux: source venv/bin/activate
```

3. Instale as dependências

```
pip install -r requirements.txt
```

4. Configure o .env

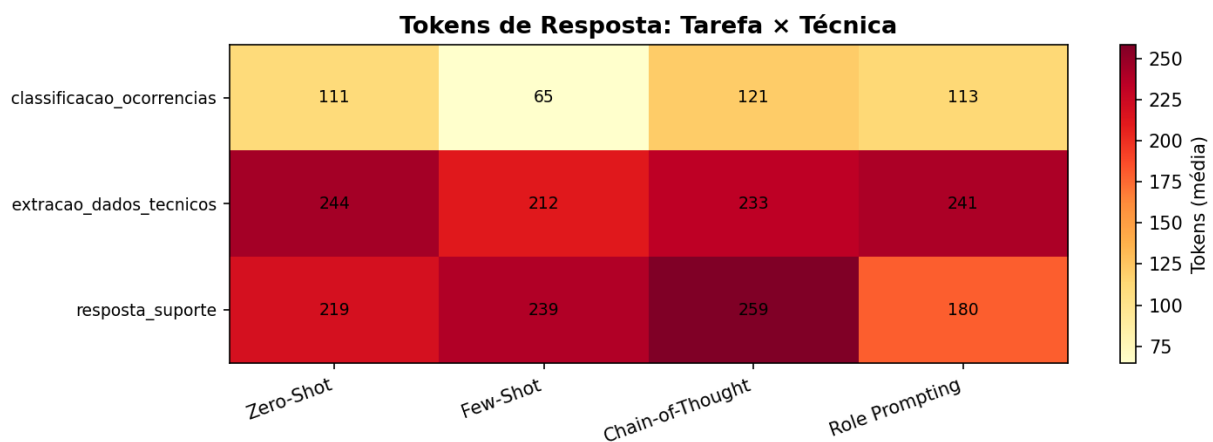
Crie um arquivo .env na raiz do projeto:

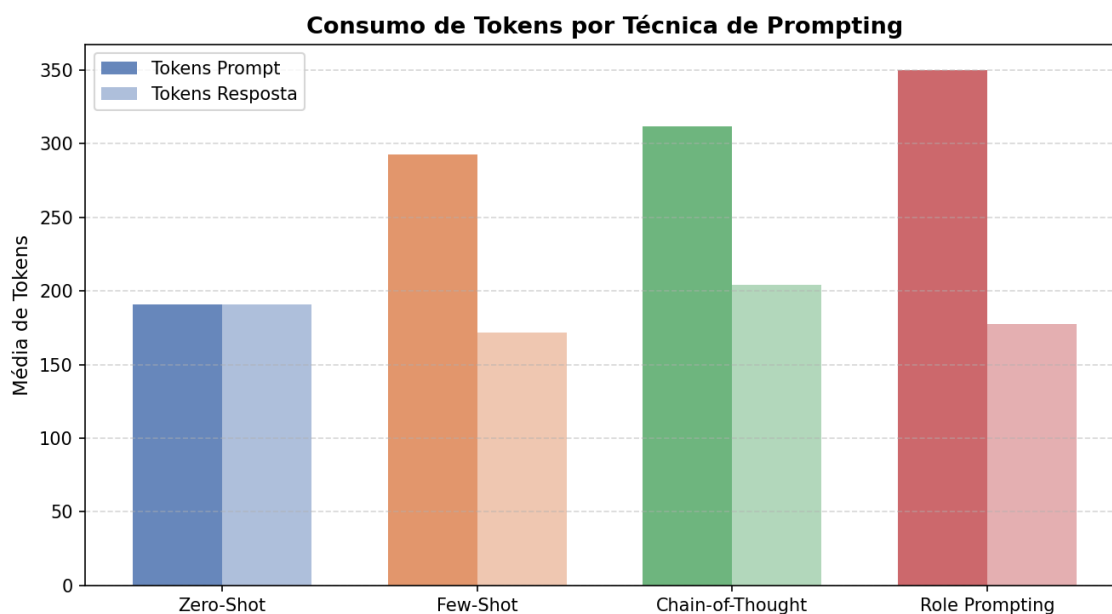
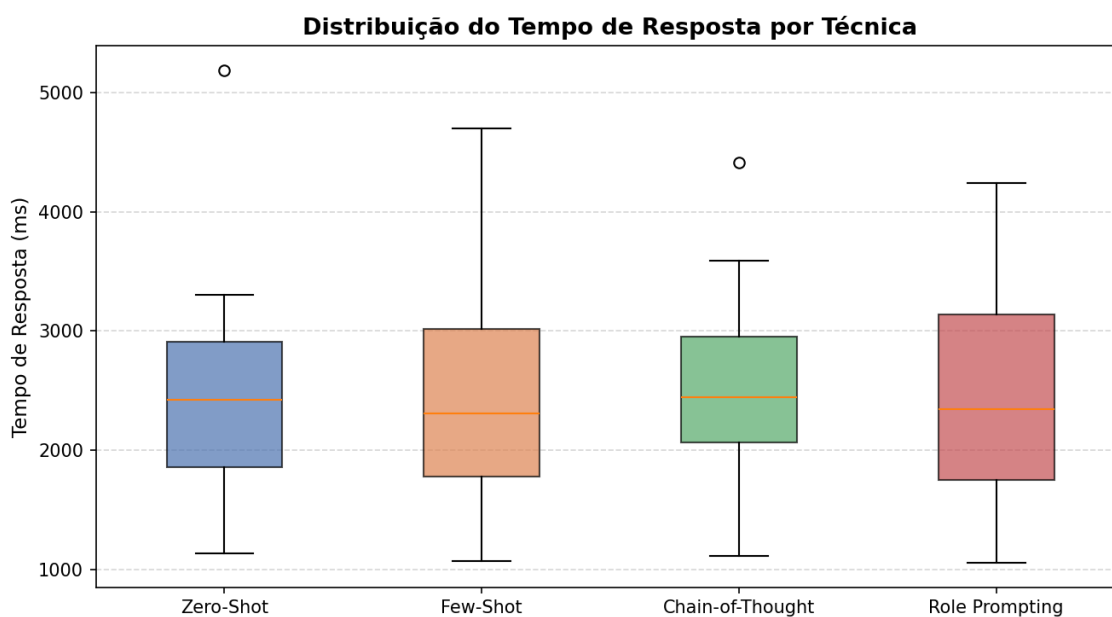
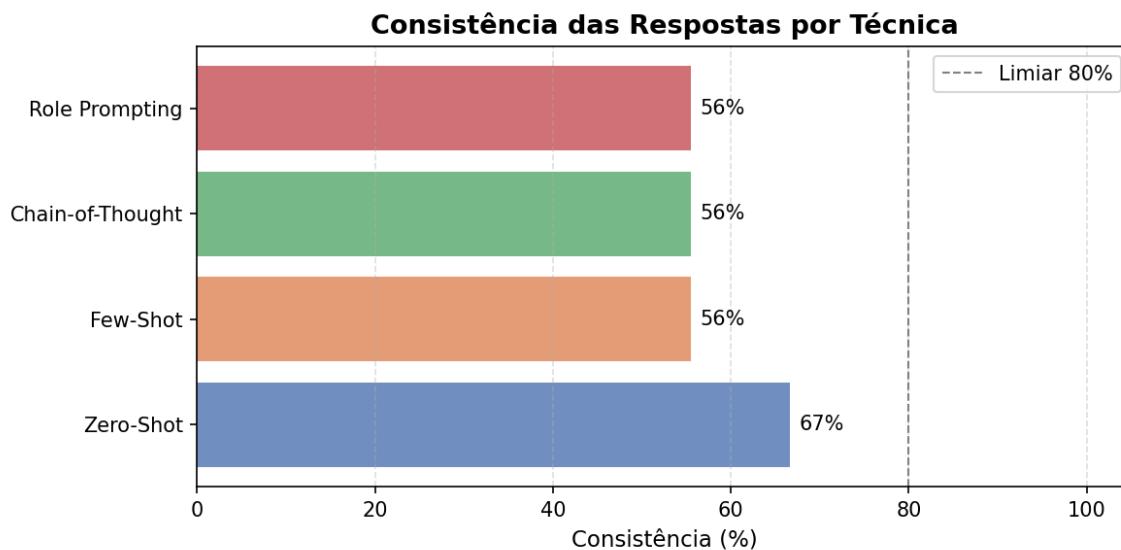
```
OLLAMA_API_KEY = sua_chave_aqui
```

5. Execute o toolkit

```
python main.py
```

8. GRÁFICOS:





9. Reflexão e Análise Crítica:

O que surpreendeu:

O mesmo input gerou classificações completamente diferentes entre as técnicas. O input "Tentei carregar meu carro, mas o cabo não encaixou no conector" resultou em FALHA_HARDWARE no Zero-Shot e Few-Shot, USO_INCORRETO no CoT e FALHA_CONECTIVIDADE no Role — todas defensáveis dependendo da perspectiva. Isso mostrou na prática que a escolha da técnica não é neutra, ela molda a interpretação do modelo.

O Chain-of-Thought consumiu até 40x mais tokens em tarefas abertas. Na classificação gerou apenas 7 tokens de resposta (uma categoria), mas na geração de resposta de suporte chegou a 278 — o mesmo modelo, a mesma técnica, comportamentos radicalmente diferentes dependendo do tipo de tarefa.

Rodando via Ollama Cloud, a latência caiu drasticamente para ~1.000–3.000ms em todas as técnicas, com distribuições muito mais uniformes entre elas — ao contrário da execução local, onde o primeiro carregamento do modelo gerava outliers de mais de 100 segundos.

A latência via Ollama Cloud ficou entre 1.000ms e 5.000ms para todas as técnicas, com mediana em torno de 2.400ms — muito mais estável e previsível do que na execução local com o modelo de 120 bilhões de parâmetros, onde variações de 10x eram comuns.

Qual técnica venceu e por que?:

O sistema recomendou automaticamente o Zero-Shot com score ponderado de 0.699 — resultado muito mais expressivo que as demais técnicas. Rodando via Ollama Cloud, o Zero-Shot se beneficiou da menor latência (~2.448ms), do menor custo de prompt e da maior consistência (67%). O algoritmo penalizou as técnicas mais elaboradas pelo alto consumo de tokens de prompt sem ganho proporcional na resposta — especialmente o Role Prompting, que usou 350 tokens de prompt para gerar apenas 176 de resposta. Isso confirma que para o domínio de eletropostos, com tarefas bem definidas e categóricas, instruções diretas superam abordagens complexas em eficiência.

O que fariam diferente:

Consistência medida por repetição real. A métrica de consistência foi calculada rodando o input representativo de cada tarefa 3 vezes por técnica. Com apenas 3 repetições, o teto matemático é 67% quando 2 de 3 respostas coincidem — o que explica os valores entre 44% e 56% observados. Para uma medição mais precisa, rodaríamos cada input pelo menos 5 vezes. O custo seria alto: o projeto já executa 99 chamadas ao modelo, e 5 repetições por input elevaria para aproximadamente 200 chamadas, mais que dobrando o tempo de execução de ~15 minutos para ~30 minutos.

Separar a recomendação por tipo de tarefa. A função `recomendar_tecnica()` calcula um score global médio entre todas as tarefas. Na prática, o Zero-Shot venceu globalmente mas a análise por tipo de tarefa poderia revelar cenários onde o CoT ou o Few-Shot seriam mais adequados, como em tarefas de geração de texto mais complexas.

Acurácia com gabarito real. O `evaluator.py` tem a função `avaliar_acuracia()` implementada, mas ela não foi integrada ao fluxo principal porque exigiria um gabarito validado por especialistas do domínio de eletropostos para cada um dos 15 inputs. Sem gabarito, a acurácia ficou fora dos gráficos. Em uma versão futura, construiríamos esse gabarito antes de rodar o toolkit.

Na execução local, a primeira chamada ao Ollama gerava outliers de mais de 100 segundos por carregar o modelo na memória. Via Ollama Cloud esse problema foi eliminado, mas em ambientes locais uma chamada de aquecimento antes do loop principal seria a solução adequada.

Domínio com mais inputs ambíguos. Inputs claros e diretos tendem a convergir para a mesma resposta em todas as técnicas, reduzindo a diferenciação. Inputs ambíguos — onde o problema pode ser classificado em mais de uma categoria — são muito mais reveladores das diferenças entre técnicas e tornariam os gráficos comparativos mais ricos.